

# Thiết Kế Kiến Trúc Hệ Thống Sinh Dữ Liệu Giả Lập (Synthetic Data Pipeline) Tối Ưu Chi Phí Và Đảm Bảo Trạng Thái Hệ Thống FANG

## Bối Cảnh Hệ Thống Và Nhận Diện Nút Thắt Cổ Chai (Bottleneck Analysis)

Trong tiến trình phát triển và hoàn thiện kiến trúc FANG v2, hệ thống được thiết kế theo mô hình "Thin Client" (FANG-Centered), nơi mọi luồng dữ liệu đầu vào và truy vấn ngữ nghĩa đều được điều phối tập trung bởi các dịch vụ lõi (AI Core) thay vì xử lý tại các giao diện người dùng.<sup>1</sup> Quá trình nạp dữ liệu (Ingestion Flow) của hệ thống này tuân thủ một chuỗi hoạt động kỹ thuật vô cùng nghiêm ngặt, bắt đầu từ việc tiếp nhận tệp hồ sơ gốc thông qua giao thức API POST /v2/ingestion/jobs, sau đó tiến hành phân tích cú pháp thông qua hệ thống 5-Tier Parser đa tầng, tiếp tục chia nhỏ văn bản theo chiến lược lai (Hybrid Chunking Strategy), và cuối cùng là quá trình nhúng vector (Embedding) trước khi lưu trữ vào cơ sở dữ liệu PostgreSQL được tích hợp tiện ích mở rộng pgvector.<sup>1</sup> Dữ liệu sau khi trải qua các lớp xử lý này sẽ được phân bổ vào hai bảng cơ sở dữ liệu cốt lõi: bảng CVPARSED đảm nhiệm vai trò lưu trữ cấu trúc JSON hoặc Markdown hoàn chỉnh để phục vụ hiển thị, và bảng AIDOCUMENTCHUNK đóng vai trò lưu trữ các mảnh văn bản kèm theo hệ thống vector ngữ nghĩa định dạng 1024 chiều nhằm phục vụ cho cơ chế tìm kiếm RAG (Retrieval-Augmented Generation).<sup>1</sup>

Tuy nhiên, bài toán hiện tại đặt ra một thách thức lớn về mặt hạ tầng khi dự án yêu cầu khả năng tự động sinh ra hơn 500 hồ sơ ứng viên (CV) và các bản mô tả công việc (Job Posting) giả lập.<sup>2</sup> Mục tiêu của tập dữ liệu này không chỉ đơn thuần là lấp đầy cơ sở dữ liệu, mà phải mô phỏng một cách chân thực sự phức tạp, độ nhiễu và những đặc trưng dị biệt của thị trường lao động công nghệ thông tin tại Việt Nam. Yêu cầu khổng lồ này ngay lập tức làm bộc lộ hai nút thắt cổ chai (bottlenecks) mang tính chí mạng đối với một dự án có ngân sách nghiên cứu hạn hẹp. Nút thắt đầu tiên là rào cản từ giới hạn lưu trữ đám mây của dịch vụ Cloudinary. Hệ thống hiện đang phụ thuộc vào phiên bản miễn phí của Cloudinary, vốn áp đặt một giới hạn cực kỳ khắt khe ở mức 25 tín dụng (credits) mỗi tháng, tương đương với khả năng lưu trữ hoặc truyền tải chỉ 25GB băng thông.<sup>1</sup> Việc đẩy hàng trăm tệp PDF lên nền tảng này sẽ nhanh chóng làm cạn kiệt tài nguyên, dẫn đến nguy cơ tê liệt toàn bộ các thao tác phát triển trong chu kỳ 30 ngày tiếp theo.<sup>6</sup>

Nút thắt thứ hai và cũng là thách thức lớn nhất liên quan đến sự bùng nổ chi phí API từ các Mô hình Ngôn ngữ Lớn (LLM). Tính đến thời điểm tháng 05/2026, thị trường trí tuệ nhân tạo chứng kiến sự thống trị của các mô hình biên giới (Frontier Models) như GPT-5.5 của OpenAI, Claude

Opus 4.7 của Anthropic và Gemini 3.1 Pro của Google.<sup>1</sup> Dù sở hữu năng lực suy luận vượt trội, chi phí vận hành của các mô hình này lại tỷ lệ thuận với sức mạnh của chúng. Việc sinh ra 500 hồ sơ chi tiết đòi hỏi một lượng token khổng lồ cho cả dữ liệu đầu vào (Input Tokens) chứa các chỉ thị phức tạp lẫn dữ liệu đầu ra (Output Tokens) định dạng JSON. Nếu không có một thiết kế đường ống sinh dữ liệu (Synthetic Data Pipeline) được tối ưu hóa một cách chiến lược từ việc chọn lựa giao thức, kỹ thuật đóng gói (Batching) cho đến định tuyến mô hình, hệ thống sẽ đối diện với rủi ro vượt quá giới hạn tài chính cho phép (thường được khống chế dưới mức 10 USD) hoặc gặp phải sự đứt gãy luồng xử lý do dữ liệu trả về bị biến dạng.<sup>1</sup> Phân tích dưới đây sẽ giải phẫu từng nút thắt, đề xuất các chiến lược kỹ thuật tinh vi và thiết lập một kịch bản giám sát toàn diện nhằm đảm bảo an toàn tuyệt đối cho hệ thống FANG trong quá trình kiểm thử toàn trình.

## Chiến Lược Lưu Trữ Định Tuyến Giả Lập (Storage & Mock Routing Strategy)

Sự phụ thuộc vào dịch vụ đám mây Cloudinary tạo ra một điểm mù trong quá trình phát triển liên tục (Continuous Development). Theo cấu trúc dịch vụ của Cloudinary, tài khoản cấp độ miễn phí (Free Tier) cung cấp 25 tín dụng hàng tháng, trong đó mỗi tín dụng có thể quy đổi thành 1GB lưu trữ, 1GB băng thông truyền tải hoặc 1.000 phép biến đổi hình ảnh (image transformations).<sup>4</sup> Trong quy trình nạp dữ liệu tiêu chuẩn của kiến trúc FANG v2, khi một file CV được giả lập và tải lên từ máy khách, nó lập tức tiêu tốn băng thông lưu trữ tại máy chủ Cloudinary.<sup>1</sup> Ngay sau đó, dịch vụ AI Core (thông qua các mô-đun như cv\_loader.py) sẽ thực hiện lệnh kéo tệp tin này về (download) để bắt đầu quá trình phân tách tại hệ thống 5-Tier Parser, tiếp tục tiêu tốn thêm một lượng băng thông tải xuống tương đương.<sup>1</sup> Nếu hệ thống thực hiện vòng lặp này cho hơn 500 hồ sơ, lượng yêu cầu HTTP và băng thông mạng sẽ bào mòn hạn mức tín dụng một cách chóng vánh, dẫn đến việc tài khoản bị khóa tạm thời và vô hiệu hóa mọi tính năng tải lên hình ảnh hay tài liệu cho toàn bộ đội ngũ phát triển.<sup>6</sup>

## Phân Tích Khả Thi Của Các Nền Tảng Lưu Trữ Đám Mây Thay Thế

Một phản xạ thiết kế thông thường để giảm tải cho dịch vụ lõi là chuyển hướng lưu trữ môi trường thử nghiệm sang các nền tảng đám mây bên thứ ba có cung cấp các gói miễn phí độc lập. Tuy nhiên, khi đánh giá sâu vào hệ sinh thái điện toán đám mây năm 2026, các giải pháp này bộc lộ những rào cản kỹ thuật nghiêm trọng đối với một đường ống nạp dữ liệu có tần suất cao:

Bảng 1: Phân tích đánh giá các dịch vụ Cloud Storage cho môi trường thử nghiệm FANG

| Nền tảng Lưu trữ Đám mây | Hạn mức Miễn phí (Free Tier 2026) | Rào cản Kỹ thuật và Chi phí Ẩn | Khả năng Tích hợp  |
|--------------------------|-----------------------------------|--------------------------------|--------------------|
| AWS S3 (Amazon)          | 5GB dung lượng                    | Phát sinh phụ phí              | Loại bỏ. Không phù |

|                                        |                                                                 |                                                                                                                                                                                                       |                                                                                                                   |
|----------------------------------------|-----------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>Web Services)</b>                   | tiêu chuẩn, 20.000 yêu cầu GET, 2.000 yêu cầu PUT. <sup>9</sup> | truy xuất dữ liệu (retrieval fees) và chi phí mạng chuyển xuất (egress). Cấu trúc phân quyền IAM (Identity and Access Management) quá phức tạp cho việc phát triển cục bộ (local dev). <sup>9</sup>   | hợp cho nhu cầu thay đổi liên tục, số lượng request lớn và rủi ro vượt chi phí do lỗi vòng lặp.                   |
| <b>Firebase Storage (Google Cloud)</b> | 5GB lưu trữ, 10GB/tháng băng thông tải xuống. <sup>12</sup>     | Băng thông tải xuống (Egress) bị giới hạn nghiêm ngặt ở mức 10GB. Nếu pipeline 5-Tier Parser liên tục kéo tệp về để kiểm thử, nguy cơ vượt ngưỡng rất cao, phát sinh phí 0.15 USD/GB. <sup>12</sup>   | Loại bỏ. Yêu cầu thiết lập bộ chứng chỉ bảo mật (Service Account) công kênh, làm giảm tốc độ phát triển mã nguồn. |
| <b>Google Drive API</b>                | 15GB (dùng chung cho toàn bộ hệ sinh thái Google Workspace).    | Giới hạn khắt khe về tốc độ yêu cầu tải xuống API (API rate limits). Drive được thiết kế cho người dùng cuối, không phải kiến trúc lưu trữ đối tượng (Object Storage) cấp doanh nghiệp. <sup>14</sup> | Loại bỏ. Không đảm bảo tốc độ băng thông nội bộ, khả năng cao gây ra lỗi Timeout cho tiến trình RAG Orchestrator. |

Sự phụ thuộc vào bất kỳ dịch vụ đám mây bên thứ ba nào trong giai đoạn thử nghiệm đều tiềm ẩn nguy cơ khóa chặt nhà cung cấp (vendor lock-in) ở mức độ cấu hình thư viện và rủi ro mạng. Hơn nữa, việc duy trì một logic mã nguồn phân mảnh (chẳng hạn dùng AWS S3 cho môi trường Development nhưng lại dùng Cloudinary cho môi trường Production) sẽ phá vỡ tính nhất quán của kiến trúc hệ thống, vi phạm nguyên tắc thiết kế mã nguồn sạch và tăng độ khó cho việc bảo trì luồng Ingestion.<sup>1</sup>

## Giải Pháp Triển Khai: Kiến Trúc Định Tuyển Giả Lập Cục Bộ (Local

## Mock Routing)

Chiến lược tối ưu nhất để vượt qua giới hạn tín dụng Cloudinary mà không cần can thiệp hay sửa đổi bất kỳ logic nghiệp vụ cốt lõi nào trong thư mục `app/services/` của dự án FANG chính là thiết lập cơ chế giả lập đường dẫn tệp (Mock File URL) kết hợp với máy chủ tĩnh cục bộ (Local Static File Server).<sup>15</sup> Kỹ thuật này hoạt động dựa trên nguyên lý đánh chặn (intercept) các lệnh gọi API phát ra từ bộ phát triển phần mềm (SDK) của Cloudinary ngay tại môi trường phát triển máy chủ nội bộ, sau đó định tuyến toàn bộ lưu lượng này về một máy chủ ảo thay vì gửi lên mạng internet.

Các hệ thống mã nguồn mở như `romajs/cloudinary-mock` hoặc các kiến trúc Express.js nội bộ cung cấp khả năng can thiệp trực tiếp vào thư viện cấu hình của ứng dụng.<sup>1</sup> Bằng cách điều chỉnh thông số cấu hình khởi tạo ngay tại tệp `app/core/config.py`, kỹ sư có thể cô lập hoàn toàn môi trường phát triển khỏi đám mây thực tế. Cụ thể, các tham số chứng thực như `cloud_name`, `api_key`, và `api_secret` sẽ được cấu hình với các chuỗi ký tự giả định (ví dụ: chuỗi 'na'), trong khi biến môi trường quan trọng nhất là `upload_prefix` sẽ được ghi đè để trở về máy chủ cục bộ (ví dụ: `http://localhost:9443` hoặc một địa chỉ IP nội bộ cấp phát cho Docker container).<sup>15</sup> Điềm cốt lõi để kỹ thuật này hoạt động trơn tru đòi hỏi phải thiết lập biến môi trường vô hiệu hóa xác minh chứng chỉ số mã hóa (ví dụ: `NODE_TLS_REJECT_UNAUTHORIZED = '0'` trong hệ sinh thái Node.js hoặc cấu hình bỏ qua xác thực SSL trong bộ thư viện yêu cầu của Python).<sup>15</sup> Thao tác này cho phép mã nguồn kết nối an toàn với máy chủ giả lập qua giao thức HTTPS mà không bị hệ điều hành ném ra các ngoại lệ từ chối bảo mật (security exceptions).

Khi tiến trình sinh dữ liệu tạo ra hàng loạt 500+ CV dưới dạng tệp PDF, các tệp này sẽ được lưu trữ vật lý trực tiếp trên ổ đĩa cứng (local disk) của môi trường phát triển. Hệ thống máy chủ giả lập cục bộ, vốn được lập trình với các cổng kết nối tương thích hoàn toàn với định dạng REST API của Cloudinary (hỗ trợ đầy đủ các phương thức HTTP GET, POST, DELETE), sẽ tiếp nhận các yêu cầu tải xuống từ luồng Ingestion và phản hồi lại cho hệ thống FANG các chuỗi dữ liệu (JSON response) có định dạng giống hệt như phản hồi từ Cloudinary thật.<sup>15</sup>

Trong cơ sở dữ liệu PostgreSQL của hệ thống, luồng xử lý đầu vào sẽ được kích hoạt bằng API POST `/v2/ingestion/jobs` như thường lệ.<sup>1</sup> Tuy nhiên, thay vì truyền vào một URL thực tế, tham số `cvSnapUrl` trong cơ sở dữ liệu sẽ chứa một đường dẫn định tuyến cục bộ, ví dụ: `http://localhost:9443/v1_1/na/image/upload/v123456789/mock_cv_001.pdf`. Cơ chế này mang lại lợi ích hệ thống vô cùng to lớn: nó cho phép các mô-đun tải tệp (`cv_loader.py`) và bộ phân tích cú pháp (`cv_parser.py`) của dự án FANG kéo tệp tin về xử lý với tốc độ tối đa của đĩa cứng ổ rắn (NVMe SSD) trên máy chủ cục bộ, loại bỏ hoàn toàn độ trễ mạng internet (network latency), đồng thời bảo toàn tuyệt đối 100% hạn mức tín dụng 25 credits của tài khoản Cloudinary dành riêng cho các tác vụ trên môi trường Production.<sup>1</sup> Hệ thống vẫn duy trì sự toàn vẹn của cấu trúc các bảng dữ liệu, các chuỗi liên kết tệp tin trong `CVPARSED` vẫn hoạt động trơn tru mà không cần tốn một byte băng thông đám mây nào.<sup>1</sup> Khi hệ thống hoàn thành quá trình kiểm thử và mã nguồn được đóng gói để triển khai lên Production, quản trị viên chỉ cần hoán đổi tập tin chứa các biến môi trường (Environment Variables), toàn bộ kiến trúc sẽ tự động chuyển đổi sang giao tiếp với máy chủ Cloudinary thực mà không cần viết lại bất kỳ dòng mã

nguồn xử lý nào.

## Giải Phẫu Thị Trường LLM 2026 Và Tối Ưu Hóa Đường Ống Dữ Liệu

Việc giải quyết rào cản lưu trữ chỉ là bước đệm đầu tiên; thách thức mang tính chất sống còn của dự án nằm ở khâu sản sinh ra một khối lượng ngữ cảnh khổng lồ. Việc tạo ra hơn 500 hồ sơ có tính chất hư cấu nhưng phải tuân thủ nghiêm ngặt các tham số định dạng JSON đòi hỏi năng lực suy luận ngôn ngữ ở mức độ rất cao, một khả năng vốn chỉ xuất hiện ở các mô hình thể hệ Frontier như GPT-5.5, Claude Opus 4.7 hoặc Gemini 3.1 Pro.<sup>1</sup> Dựa trên các báo cáo phân tích thị trường công nghệ (P1 và P4) tại Việt Nam cập nhật đến tháng 05/2026, chiến lược lựa chọn API là yếu tố cốt lõi để đảm bảo hệ thống không bị phá vỡ về mặt tài chính lẫn kiến trúc kỹ thuật.<sup>1</sup>

### Bức Tranh Toàn Cảnh Về Proxy Lậu Và Sự Đứt Gãy Hệ Thống

Trước sức ép về chi phí API chính thống cực kỳ đắt đỏ (khoảng 5.00 USD cho mỗi 1 triệu token đầu vào và 30.00 USD cho mỗi 1 triệu token đầu ra đối với GPT-5.5<sup>8</sup>), thị trường công nghệ tại Việt Nam đã chứng kiến sự trỗi dậy của một hệ sinh thái ngầm chuyên phân phối lại API (API Reselling/Proxying) với các nền tảng tiêu biểu như 9router (tại tên miền devgovietnam.io.vn), claudeprovn, và krouter.<sup>1</sup> Các nền tảng này thu hút các dự án ngân sách thấp bằng cách cung cấp quyền truy cập vào các mô hình AI đỉnh cao với mức phí chỉ bằng một phần nhỏ giá niêm yết của nhà phát hành.

Về mặt kiến trúc, các hệ thống proxy này hoạt động dựa trên cơ chế Định tuyến Trung gian (LLM API Gateway) kết hợp với kỹ thuật Đầu cơ Tài nguyên (Resource Arbitrage).<sup>1</sup> Người dùng hệ thống lậu không kết nối trực tiếp với OpenAI hay Anthropic, mà gửi tải trọng (payload) đến một máy chủ trung gian (Reverse Proxy). Tại đây, hệ thống sẽ thực hiện thao tác gom nhóm tài khoản (Account Pooling), gỡ bỏ các mã khóa xác thực giả lập của người dùng và bí mật tiêm (inject) một API Key chính thống được lấy từ một kho chứa (Pool) khổng lồ trước khi chuyển tiếp yêu cầu đến nền tảng gốc.<sup>1</sup> Nguồn cung cấp các khóa API chính thống này thường đến từ các hoạt động vi phạm chính sách bảo mật, bao gồm việc lạm dụng hạn ngạch miễn phí (Free Tier Farming) thông qua hàng vạn tài khoản rác, lạm dụng tín dụng đám mây (Cloud Credit Abuse) thông qua các pháp nhân khởi nghiệp ảo để nhận tài trợ, hoặc thậm chí là khai thác lỗ hổng bảo mật máy chủ (LLMjacking) và sử dụng thẻ tín dụng đánh cắp để thanh toán dịch vụ.<sup>1</sup>

Mặc dù chi phí là một yếu tố vô cùng hấp dẫn đối với dự án sinh viên, song theo như lưu ý từ kỹ sư hệ thống, độ ổn định và chất lượng cấu trúc dữ liệu mới là ưu tiên tối thượng. Đối với một đường ống sinh dữ liệu tự động, việc sử dụng các dịch vụ proxy lậu mang lại rủi ro kỹ thuật ở mức độ thảm họa. Kiến trúc chia sẻ tài nguyên này gặp phải vấn đề xung đột giới hạn tỷ lệ (Rate Limit Contention) cực kỳ trầm trọng; khi hàng trăm người dùng lậu cùng đẩy các tác vụ sinh văn bản nặng nề, kho chứa API sẽ liên tục chạm ngưỡng bảo vệ của OpenAI, dẫn đến việc kết nối bị từ chối với mã lỗi HTTP 429 (Too Many Requests) làm tê liệt hoàn toàn luồng hoạt động.<sup>1</sup> Nghiêm trọng hơn, các máy chủ proxy rẻ tiền này thường xuyên bị định cấu hình

sai về kích thước bộ nhớ đệm (buffer) hoặc thiết lập thời gian chờ (timeout) quá ngắn. Hậu quả là các dòng dữ liệu đang truyền tải (data streams) bị hệ thống ngắt kết nối giữa chừng (Truncated Streams).<sup>1</sup>

Sự cố văn bản bị cắt cụt có thể được con người tự suy luận để hiểu khi đọc văn xuôi, nhưng đối với hệ thống phần mềm ép kiểu dữ liệu của FANG, một chuỗi JSON trả về bị thiếu đi dấu đóng ngoặc } hay dấu phẩy phân tách trường dữ liệu sẽ ngay lập tức gây ra lỗi giải mã (JSON Decode Error), làm sụp đổ toàn bộ tiến trình nạp Ingestion.<sup>1</sup> Thêm vào đó, việc định tuyến dữ liệu qua máy chủ trung gian không được chứng thực tương đương với việc mở cửa cho một cuộc tấn công xen giữa (Man-in-the-Middle - MitM). Các bản mô tả công việc, kịch bản huấn luyện và mã nguồn cấu trúc gửi đi dưới dạng văn bản thuần (Plain Text Prompts) có thể bị thu hoạch (Data Harvesting) ngầm, hoặc nguy hiểm hơn, bị kẻ gian sửa đổi để tiêm nhiễm mã độc vào nội dung phản hồi.<sup>1</sup> Chính vì những rào cản về độ ổn định cấu trúc và lỗ hổng bảo mật dữ liệu, chiến lược phát triển hệ thống kiên quyết loại trừ phương án sử dụng proxy lậu và chuyển hướng sang việc khai thác tối ưu hóa các giao thức API chính thống của những mô hình tiết kiệm.<sup>1</sup>

**Tối Ưu Hóa Chi Phí Bằng Kỹ Thuật Batching Cấu Trúc (Structured Output Batching)**

Việc từ bỏ thị trường chợ đen đồng nghĩa với việc dự án phải đối mặt với áp lực tài chính thực sự. Nếu thực hiện gọi API từng lần một một cách lè tẻ để sinh ra 500 CV, lượng token dư thừa cấu thành từ các chỉ thị ngữ cảnh (System Prompt) liên tục bị lặp lại trong mỗi HTTP Request sẽ làm bùng nổ chi phí.<sup>1</sup> Do đó, kiến trúc tối ưu hóa bắt buộc phải áp dụng kỹ thuật xử lý hàng loạt (Batching) thông qua các giao thức hàm ý định dạng cấu trúc (Structured Outputs).

Bảng 2: So sánh cấu trúc chi phí API chính thống phục vụ kỹ thuật Batching (Tháng 05/2026)

| Mô hình Ngôn ngữ Lớn | Vị thế Công nghệ                | Chi phí Input (Mỗi 1M Token)                                                  | Chi phí Output (Mỗi 1M Token)                               | Lợi thế Kỹ thuật cho Batching                                                                                                       |
|----------------------|---------------------------------|-------------------------------------------------------------------------------|-------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| GPT-5.5              | Lõi suy luận cấp cao (Frontier) | 5.00 USD (Tiêu chuẩn) / 2.50 USD (Batch API) / 0.50 USD (Cached) <sup>8</sup> | 30.00 USD (Tiêu chuẩn) / 15.00 USD (Batch API) <sup>8</sup> | Độ chính xác tuyệt đối trong suy luận đa bước. Chế độ Batch API giảm 50% chi phí nhưng xử lý phi đồng bộ (độ trễ 24h). <sup>8</sup> |



|                              |                                     |                                                                       |                                                                              |                                                                                                                                    |
|------------------------------|-------------------------------------|-----------------------------------------------------------------------|------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <b>Gemini 3.1 Pro</b>        | Cấp độ chuyên gia đa phương thức    | Tương đương hệ Frontier                                               | Tương đương hệ Frontier                                                      | Khả năng kiểm soát thuộc tính JSON nghiêm ngặt (Implicit property ordering) thông qua Pydantic. <sup>7</sup>                       |
| <b>Gemini 3.1 Flash-Lite</b> | Tối ưu hóa vi mô, tốc độ siêu nhanh | Miễn phí (trong hạn mức Free Tier 15 RPM, 1.000.000 TPM) <sup>1</sup> | Miễn phí (trong hạn mức Free Tier 15 RPM, 65.536 Output Tokens) <sup>1</sup> | "Ngựa thồ" sản xuất dữ liệu của hệ thống. Hỗ trợ đầy đủ Structured Outputs và cửa sổ ngữ cảnh lên tới 1 triệu token. <sup>20</sup> |
| <b>GPT-5.4-mini</b>          | Dự phòng Lite (Fallback Judge)      | 0.75 USD <sup>18</sup>                                                | ~4.50 USD                                                                    | Cực rẻ, được sử dụng làm chốt chặn thẩm định định dạng (Format Judge) khi các mô hình miễn phí gặp ảo giác. <sup>1</sup>           |

Để thực thi kỹ thuật Batching, kỹ sư dữ liệu sẽ kiến trúc một System Prompt duy nhất nhưng mang sức tải khổng lồ, yêu cầu mô hình (điển hình là Gemini 3.1 Flash-Lite) sinh ra cùng một lúc một mảng JSON (JSON array) chứa từ 10 đến 20 đối tượng hồ sơ giả lập trong một lần gọi mạng. Việc đóng gói dữ liệu này mang lại hiệu quả kinh tế học token (Token Economics) vô cùng to lớn: toàn bộ phần văn bản chứa các chỉ thị nguyên tắc (System Instructions) được chia sẻ chung cho 20 hồ sơ, triệt tiêu sự lãng phí token đầu vào.<sup>1</sup> Đối với Gemini 3.1 Flash-Lite, với khả năng hỗ trợ tối đa lên đến 65.536 token đầu ra cho mỗi truy vấn, mô hình này hoàn toàn dư dả không gian để sinh ra mảng JSON chứa hàng chục CV mà không bị tràn bộ đệm.<sup>20</sup>

Chìa khóa để đảm bảo mảng JSON sinh ra không bị sai lệch kiểu dữ liệu nằm ở việc áp dụng công nghệ Structured Outputs. Khác với JSON Mode thông thường vốn chỉ đảm bảo định dạng văn bản đúng chuẩn JSON, hệ thống FANG sẽ sử dụng thư viện Pydantic (trong Python) hoặc Zod (trong JavaScript) để định nghĩa trước một lược đồ dữ liệu chặt chẽ (JSON Schema).<sup>19</sup> Các mô hình tiên tiến như Gemini 3.1 Flash-Lite đã được Google cập nhật khả năng ép buộc đầu ra (constrain output) bám sát tuyệt đối vào lược đồ này, đồng thời hỗ trợ các từ khóa phức tạp như \$ref cho cấu trúc đệ quy, hay anyOf cho các kiểu dữ liệu liên hiệp (Unions).<sup>19</sup> Việc ép

buộc mô hình tuân thủ trật tự thuộc tính (Implicit property ordering) giúp triệt tiêu hoàn toàn tỷ lệ lỗi phân giải khi hệ thống cv\_parser.py của FANG tiến hành bóc tách và chèn thông tin vào cơ sở dữ liệu CVPARSED.<sup>1</sup> Bằng cách khai thác hạn ngạch miễn phí 15 RPM của Gemini Flash-Lite và kỹ thuật Batching 20 hồ sơ/lượt, hệ thống có thể dễ dàng sinh ra hơn 1.000 CV mỗi ngày với chi phí API hoàn toàn bằng không.<sup>1</sup>

## **Nghệ Thuật Prompt Engineering: Giả Lập Thị Trường Nhân Sự Công Nghệ Việt Nam**

Chất lượng của một tập dữ liệu giả lập không nằm ở khối lượng, mà nằm ở độ chân thực và khả năng tạo ra các kịch bản học búa để thử thách các dịch vụ hệ thống. Mục tiêu cốt lõi của việc sinh 500+ hồ sơ là để kiểm thử sức chịu đựng của hệ thống 5-Tier Parser đa tầng (từ mô hình Lite đến Pro), cũng như huấn luyện thuật toán đối nghịch (contrastive learning) cho cơ chế đánh giá RAG của hệ thống xếp hạng NMAlex.<sup>1</sup> Do đó, Prompt Engineering cần được tinh chỉnh để tạo ra một môi trường giả lập (Hybrid Generation) pha trộn giữa các đồ thị kỹ năng logic (Skill Graphs) và các lớp nhân vật (Persona-driven Overlay) mang đậm đặc tính của thị trường lao động IT Việt Nam.<sup>1</sup>

### **1. Sự Đa Dạng Trong Dữ Liệu Thực Thể Bản Địa (Entity Localization)**

Hồ sơ giả lập phải phản ánh được sự không đồng nhất trong hệ thống giáo dục và văn hóa đặt tên tại Việt Nam. System Prompt phải đưa ra chỉ thị nghiêm ngặt cho LLM lồng ghép các cơ sở đào tạo công nghệ thông tin trọng điểm dưới nhiều dạng biến thể khác nhau. Ví dụ, cùng một trường đại học phải được xuất hiện dưới các dạng: "Đại học Bách Khoa Hà Nội", "Hanoi University of Science and Technology", hoặc viết tắt là "HUST"; tương tự đối với "Đại học Khoa học Tự nhiên TP.HCM" (HCMUS, KHTN) hay "Học viện Công nghệ Bưu chính Viễn thông" (PTIT).<sup>23</sup> Sự đa dạng trong cách ghi nhận danh xưng này sẽ tạo ra những rào cản từ vựng, buộc các mô hình AI trong luồng phân tích thực thể (Named Entity Recognition) của FANG phải học cách chuẩn hóa (normalization) dữ liệu về một nhãn thống nhất. Cấu trúc tên ứng viên cũng cần được rải rác đan xen giữa tên có đầy đủ dấu thanh tiếng Việt, tên không dấu, và các cấu trúc họ-tên phức tạp đặc trưng của văn hóa bản địa.

### **2. Sự Lệch Pha Trạng Thái Tâm Lý Hành Vi (Behavioral Personas)**

Để mô phỏng sự hỗn loạn thực tế của thị trường tuyển dụng, Prompt phải xây dựng các cấu hình nhân vật (personas) chứa đựng các yếu tố dị thường. Điển hình nhất là thiết lập nhóm nhân vật "Fresher ảo tưởng sức mạnh" chiếm khoảng 20-30% tổng số hồ sơ. Đặc trưng của nhóm này là sở hữu thâm niên thực tế rất ngắn (chỉ từ 3 đến 6 tháng thực tập hoặc mang danh nghĩa thực hiện dự án tốt nghiệp) nhưng lại đòi hỏi mức lương kỳ vọng (Expected Salary) cao ngất ngưỡng, đồng thời khai khống một danh sách dài các kỹ năng kiến trúc sư hệ thống phức tạp (như Kubernetes, Distributed Systems, Microservices) ở mức độ "Thành thạo" (Proficient).<sup>26</sup> Ở thái cực ngược lại, hệ thống cần sinh ra nhóm hồ sơ "Overqualified" (thừa thâm niên) – những kỹ sư cao cấp với 8 năm kinh nghiệm nhưng lại ứng tuyển vào các vị trí Junior. Những dữ liệu bất đối xứng này không phải là lỗi sinh dữ liệu, mà là nguyên liệu hạt nhân để kích hoạt



các cơ chế thuật toán quan trọng trong hệ thống xếp hạng NMAlex, bao gồm tính năng Phạt Thâm Niên (Seniority Penalty Asymmetric Buffer) và Điều chỉnh Lương (Salary Adjustment).<sup>1</sup> Sự xuất hiện của các hồ sơ này giúp kiểm tra khả năng của hệ thống RAG trong việc đánh giá mức độ tin cậy của ứng viên thay vì chỉ so khớp từ khóa cơ học.<sup>1</sup>

### 3. Tiêm Nhiễu Cấu Trúc Và Định Dạng (Formatting Noise Injection)

Luồng xử lý đầu vào của FANG phụ thuộc rất lớn vào chiến lược phân mảnh lai (Hybrid Chunking Strategy), vốn cắt nhỏ văn bản dựa trên các thẻ tiêu đề (heading) của định dạng Markdown.<sup>1</sup> Do đó, Prompt phải cố tình tạo ra sự hỗn loạn trong định dạng văn bản (Formatting Noise) để đẩy bộ phân tích 5-Tier Parser đến giới hạn chịu đựng. Cụ thể, LLM sẽ được yêu cầu:

- **Hỗn hợp ngôn ngữ (Code-Switching):** Chèn các thuật ngữ tiếng Anh chuyên ngành (như Framework, CI/CD, Deployment pipeline) lộn xộn vào giữa các câu trúc tiếng Việt, phản ánh thói quen giao tiếp thực tế của kỹ sư phần mềm Việt Nam.
- **Lỗi hệ thống đánh máy:** Mô phỏng các sai sót chính tả do cơ chế gõ Telex/VNI gây ra (ví dụ: gõ sai dấu thanh, lỗi khoảng trắng do bộ mã Unicode không chuẩn).<sup>27</sup> Những khiếm khuyết ngôn ngữ này sẽ thử thách khả năng chịu đựng của mô hình nhúng (Embedding Model).
- **Vỡ định dạng cấu trúc:** Yêu cầu mô hình sinh ra các chuỗi văn bản đại diện cho cấu trúc bảng biểu PDF bị vỡ, hoặc các khoảng trắng không đồng nhất (Tab/Space hỗn loạn). Việc này giả lập hiện trạng các trình đọc PDF (PDF Parsers) cấp thấp gặp lỗi khi trích xuất dữ liệu đa cột, tạo ra một bài kiểm tra hạng nặng cho cơ chế ProTierGate của FANG trong việc ra quyết định có nên "leo thang" (Escalate) đẩy hồ sơ này lên các mô hình đắt tiền như GPT-5.4 để sửa lỗi hay không.<sup>1</sup>

Với tỷ lệ kiểm soát 180 hồ sơ nhiễu cho mỗi 1 quyết định tuyển dụng hợp lệ (tỷ lệ 180:1), chiến lược này sẽ đào tạo cho bộ não AI của hệ thống cách lợi ngược dòng thông tin rác, qua đó chất lọc được những ứng viên thực sự tiềm năng trong một ma trận dữ liệu phi chuẩn.<sup>1</sup>

## Quan Trắc Hệ Thống (Observability) Và Kịch Bản Triển Khai Kiểm Thử

Triển khai một đường ống sinh dữ liệu lớn theo phương pháp "nhắm mắt làm bừa" (blind doing) – tức là gọi API và kỳ vọng nó hoàn thành hàng trăm hồ sơ trong một lượt – tiềm ẩn rủi ro sụp đổ thảm họa. Nếu kỹ thuật Prompt bị thiết kế sai lệch dẫn đến mô hình bị ảo giác (hallucination) tạo ra dữ liệu hỏng, hoặc hệ thống vi phạm giới hạn token dẫn đến việc bị khóa API, toàn bộ thời gian và ngân sách dự án sẽ bốc hơi vô ích. Kèm theo đó, kỹ sư sẽ phải đối mặt với thảm họa làm sạch cơ sở dữ liệu (Database Purging) để xóa bỏ các tệp tin lỗi. Do đó, việc thiết lập một mạng lưới giám sát toàn diện (Observability) kết hợp với phương pháp triển khai cuốn chiếu theo từng giai đoạn (Monitored Phased Rollout) là điều kiện bắt buộc.

### Kiến Trúc Giám Sát LLM Thời Gian Thực (LLM Observability)

## Architecture)

Hệ thống cần tích hợp các công cụ quan trắc mã nguồn mở (Open Source) siêu nhẹ vào môi trường Python thông qua việc bao bọc (wrapping) các lời gọi API. Các công cụ tiên tiến như LiteLLM, OpenLIT hoặc Evidently AI đóng vai trò như một lớp cổng giao tiếp (AI Gateway) ghi nhận mọi tương tác của LLM với độ trễ tính toán chỉ khoảng 2ms.<sup>30</sup> Cơ chế này mang lại khả năng giám sát ở bốn tầng cốt lõi:

- **Vết tích vòng đời (Tracing):** Chụp lại và trực quan hóa toàn bộ hành trình của yêu cầu, từ cụm từ Prompt sơ khai cho đến khi chuỗi JSON hoàn chỉnh được mô hình trả về.
- **Đo lường dung lượng (Metrics Tracking):** Giám sát liên tục số lượng token tiêu thụ trên mỗi yêu cầu, theo dõi độ trễ mạng (Latency), và đặc biệt là đếm tần suất máy chủ Google hoặc OpenAI ném ra các cảnh báo giới hạn lưu lượng (Rate Limit hit rate).
- **Giám sát Tài chính (Cost Monitoring):** Tự động quy đổi lượng token tiêu thụ ra giá trị tiền tệ thực tế theo thời gian thực. Bảng điều khiển (Dashboard) sẽ đối chiếu mức chi tiêu này với các mốc ngân sách kế hoạch (cấp 2 USD, 3 USD, 5 USD, hay 10 USD) để đảm bảo tốc độ "đốt tiền" (burn rate) luôn nằm trong tầm kiểm soát.<sup>1</sup>
- **Thẩm định Chất lượng Tự động (LLM-as-a-judge):** Thiết lập một luồng rẽ nhánh, sử dụng mô hình siêu nhẹ (như GPT-5.4-nano) để đóng vai trò "giám khảo định dạng". Mô hình này sẽ kiểm tra tính toàn vẹn của cấu trúc JSON và đánh giá độ đa dạng của văn phong trước khi quyết định ghi dữ liệu vào bảng CVPARSED.

## Kịch Bản Triển Khai Kiểm Thử Ba Giai Đoạn (Safe Rollout Scenario)

Việc vận hành quá trình sinh dữ liệu đòi hỏi một khoảng thời gian giám sát tập trung từ 1 đến 3 ngày, hoặc một phiên vận hành lý tưởng kéo dài vài giờ liên tục, chia làm ba giai đoạn bảo vệ nghiêm ngặt:

**Giai đoạn 1: Khởi động khô và Lấy mẫu hạt giống (Dry Run & Smoke Testing)** Hệ thống sẽ khai thác các kịch bản kiểm thử nền tảng hiện có của thư mục dự án FANG, tiêu biểu là việc gọi tập lệnh `smoke_tests/test_e2e_pipeline.py`.<sup>1</sup> Đường ống sẽ chỉ định tuyến gửi đi một yêu cầu sinh thử nghiệm duy nhất chứa từ 1 đến 3 bộ hồ sơ giả lập. Ngay khi JSON được trả về, quy trình sẽ tự động tạm ngưng để các kỹ sư thu thập thông số về thời gian phản hồi, độ tiêu hao token và kiểm chứng tính hợp lệ của cấu trúc lược đồ Pydantic. Các hồ sơ mẫu này sẽ được ép chạy thử qua toàn bộ luồng Ingestion Flow để đánh giá xem mô-đun `cv_parser.py` và cơ chế chia nhỏ văn bản (Chunking) có bẻ gãy tài liệu chính xác thành các bản ghi AIDOCUMENTCHUNK hay không.<sup>1</sup> Nhật ký hệ thống (System Logs) sẽ được quét tự động để đảm bảo không xảy ra ngoại lệ (exception) khi tương tác với PostgreSQL qua tiện ích pgvector.<sup>1</sup>

**Giai đoạn 2: Bùng nổ Vi mô và Thẩm định Chất lượng (Micro-burst & Quality Assurance)** Sau khi vượt qua thử nghiệm lỗi, quy mô sẽ được khuếch đại lên một lô (batch) khoảng 50 CV và 10 bản mô tả công việc (Job Postings). Toàn bộ lưu lượng này được định tuyến qua mô hình Gemini 3.1 Flash-Lite nhằm mô phỏng "Cấu hình Quick" – tức là cấu hình thử nghiệm tiêu tốn 0

USD.<sup>1</sup> Trong giai đoạn này, các thông số cảnh báo tự động (Alerting) sẽ được thiết lập trên nền tảng giám sát (chẳng hạn Datadog hoặc Grafana) nhằm kích hoạt thông báo khi tỷ lệ lỗi vượt quá giới hạn.<sup>33</sup> Dữ liệu sau khi sinh sẽ được đẩy qua luồng nạp thực tế để kiểm tra hành vi của hệ thống ProTierGate. Kỹ sư sẽ quan sát xem tỷ lệ các CV chứa nhiều cấu trúc có thực sự ép hệ thống phải "leo thang" gọi các mô hình trả phí (như GPT-5.4) để cứu vãn lỗi định dạng hay không.<sup>1</sup>

**Giai đoạn 3: Triển khai Quy mô lớn phi đồng bộ (Full-scale Asynchronous Execution)** Chỉ khi cả hai giai đoạn trên vượt qua mọi rào cản kiểm định chất lượng, kịch bản sinh dữ liệu gốc mới được kích hoạt cho toàn bộ 500+ hồ sơ. Quy trình này sẽ vận hành trong nền, áp dụng thuật toán lùi bước theo hàm mũ (exponential backoff) để tự động "ngủ" và xử lý duy trì trạng thái khi nền tảng đám mây phản hồi mã lỗi 429 do vượt quá giới hạn 15 RPM.<sup>1</sup> Sự kiên nhẫn trong việc thiết lập giới hạn thời gian (rate limits) sẽ đảm bảo toàn bộ tập dữ liệu khổng lồ này được sinh ra mà không gặp bất kỳ đứt gãy nào về cấu trúc phân mảnh.

## Đánh Giá Toàn Trình (End-to-End Evaluation) Và Chuẩn Bị Dữ Liệu Gán Nhãn (Labeling)

Mục đích cuối cùng của tập dữ liệu 500+ hồ sơ này không dừng lại ở việc kiểm tra khả năng nạp dữ liệu (Ingestion), mà là một bài kiểm tra mức độ chịu đựng cho toàn bộ kiến trúc FANG, từ giao diện trò chuyện (Chat) cho đến thuật toán xếp hạng phức tạp của NMAlex (Network Matching Artificial Intelligence).<sup>1</sup> Điều này đồng nghĩa với việc, trong quá trình cấu trúc Prompt sinh dữ liệu, mô hình phải được chỉ thị chèn thêm các trường dữ liệu ẩn (hidden fields) phục vụ riêng cho mục đích gán nhãn (labeling) và huấn luyện (training).

Theo tài liệu thiết kế chiến lược của NMAlex, hệ thống xếp hạng đánh giá các ứng viên dựa trên đa chiều không gian, đòi hỏi cấu trúc JSON của mỗi hồ sơ giả lập phải chứa trước các nhãn tham chiếu (Ground Truth Labels) bao gồm <sup>1</sup>:

- **Nhân Kỹ năng (Skill Mapping):** Dữ liệu giả lập cần phải chứa các kỹ năng nằm trong tập danh mục đóng (Closed-World catalog) để hệ thống so khớp chính xác (Exact Overlap), đồng thời chứa các kỹ năng nằm ngoài danh mục (Open-World) để thử thách khả năng tính toán độ tương đồng mờ (Fuzzy Overlap) qua vector của mô hình.<sup>1</sup>
- **Thâm niên và Hình phạt (Seniority Penalty):** Mọi hồ sơ sinh ra phải bao gồm trường thông tin thể hiện rõ số năm kinh nghiệm thực tế. Điều này cho phép các kỹ sư kiểm chứng xem cơ chế "Vùng đệm bất đối xứng" (Asymmetric Buffer) có phạt nặng những ứng viên thiếu kinh nghiệm và nương tay với những ứng viên thừa thâm niên (overqualified) đúng như logic toán học định sẵn hay không.<sup>1</sup>
- **Điều chỉnh Lương (Salary Adjustment):** Trường mức lương kỳ vọng phải được gán nhãn rõ ràng để đánh giá tính năng phạt điểm khi mức lương công việc quá thấp, hoặc trần thưởng (salary\_bonus\_cap) khi mức lương vượt kỳ vọng.<sup>1</sup>
- **Nhãn Địa lý và Ngôn ngữ (Geo & Language Labels):** Mô hình sinh dữ liệu phải tạo ra sự không đồng nhất, chẳng hạn liệt kê chứng chỉ tiếng Nhật "N3" hoặc "IELTS 7.5", buộc

hệ thống LLM Mapper của NMAlex phải hoạt động để chuẩn hóa các giá trị này về thang đo 5 bậc nguyên thủy, cũng như ánh xạ địa danh cũ về mã định danh tỉnh/thành phố hiện hành.<sup>1</sup>

Quá trình kiểm thử toàn trình yêu cầu toàn bộ 500 CV và Job Postings giả lập này đi qua toàn bộ luồng xử lý thực tế của hệ thống: từ việc phân tích cú pháp (Parsing) tại hệ thống 5 tầng, chia nhỏ văn bản (Chunking), cho đến việc biến đổi văn bản thành các vector số học (Embedding).<sup>1</sup> Câu hỏi về chi phí tài nguyên (Token Cost) cho toàn bộ chu trình kiểm định này là một lo ngại hoàn toàn chính đáng, đặc biệt khi dự án bị giới hạn nghiêm ngặt bởi hạn mức tín dụng 5 USD của OpenAI.<sup>1</sup>

Tuy nhiên, với kiến trúc Định tuyến Đa mô hình (Multi-model Routing) được thiết kế hiện tại, chi phí đánh giá toàn trình đã được tối ưu hóa về mức an toàn tuyệt đối. Giai đoạn phân tích cú pháp và trích xuất dữ liệu thô (Parsing) của toàn bộ 500 hồ sơ sẽ được xử lý hoàn toàn miễn phí nhờ sự gánh vác của "ngựa thồ" Gemini 3.1 Flash-Lite.<sup>1</sup> Các hồ sơ lỗi định dạng sẽ được ProTierGate xử lý nhưng tỷ lệ này sẽ được kiểm soát ở mức độ thấp. Sau khi hệ thống băm nhỏ các hồ sơ này thành khoảng 1.500 mảnh văn bản (ước tính 3 chunks cho mỗi CV)<sup>1</sup>, tổng số lượng token cần nhúng (Embedding Tokens) sẽ dao động trong khoảng 400.000 đến 500.000 token.<sup>1</sup>

Với việc hệ thống tiêu chuẩn hóa sử dụng mô hình text-embedding-3-small của OpenAI, vốn có đơn giá cực kỳ rẻ ở mức 0.02 USD cho mỗi 1 triệu token<sup>1</sup>, tổng chi phí để chuyển đổi toàn bộ tập dữ liệu 500 CV giả lập thành cơ sở dữ liệu vector chỉ tiêu tốn chưa đến 0.01 USD. Khoản chi phí siêu vi mô này chỉ chiếm một phần vài nghìn trong quỹ tín dụng 5 USD của hệ thống, cho phép kỹ sư liên tục thực thi kịch bản dọn dẹp và nạp lại dữ liệu (Reset and Seed Database) thông qua tập lệnh scripts/reset\_and\_seed\_db.py mà không phải bận tâm về sự cạn kiệt tài chính.<sup>1</sup> Sự cộng hưởng giữa chi phí nhúng siêu rẻ và hạn ngạch xử lý miễn phí khổng lồ của Google API đã tạo ra một hệ thống sinh dữ liệu và kiểm thử giả lập hoàn hảo, cho phép đội ngũ NMAlex thỏa sức chạy các vòng lặp Benchmark liên tục, tính toán các chỉ số nDCG@10, Precision hay Recall@20 mà vẫn giữ vững triết lý thiết kế tối giản và bền vững về chi phí.<sup>1</sup>

## **Nguồn trích dẫn**

1. input\_processing\_guide.md
2. Synthetic Data Generation APIs Market Outlook 2026-2034, truy cập vào tháng 5 10, 2026, <https://www.intelmarketresearch.com/synthetic-data-generation-apis-market-28272>
3. 3rd Synthetic Data for Computer Vision - CVPR 2026 | Synthetic Data for Computer Vision, truy cập vào tháng 5 10, 2026, <https://syndata4cv.github.io/>
4. Cloudinary Pricing 2026: Plans, Costs & Real Expenses - CheckThat.ai, truy cập vào tháng 5 10, 2026, <https://checkthat.ai/brands/cloudinary/pricing>
5. Pricing and Plans - Cloudinary, truy cập vào tháng 5 10, 2026, <https://cloudinary.com/pricing>

6. What happens if I exceed the usage limits of my account's plan? - Cloudinary Support, truy cập vào tháng 5 10, 2026, <https://support.cloudinary.com/hc/en-us/articles/202521702-What-happens-if-I-exceed-the-usage-limits-of-my-account-s-plan>
7. Gemini 3 Developer Guide - generateContent API, truy cập vào tháng 5 10, 2026, <https://ai.google.dev/gemini-api/docs/gemini-3>
8. GPT-5.5 Pricing: How Much Does It Cost in 2026? - CometAPI - All AI Models in One API, truy cập vào tháng 5 10, 2026, <https://www.cometapi.com/how-much-is-gpt-5-5/>
9. Cloud & AI Storage Pricing Comparison 2026: AWS, Azure, GCP, OCI - Finout, truy cập vào tháng 5 10, 2026, <https://www.finout.io/blog/cloud-storage-pricing-comparison>
10. Cloud Storage pricing - Google Cloud, truy cập vào tháng 5 10, 2026, <https://cloud.google.com/storage/pricing>
11. AWS S3 Pricing Guide: Mastering Cloud Storage Costs in 2026 - Hyperglance, truy cập vào tháng 5 10, 2026, <https://www.hyperglance.com/blog/aws-s3-pricing-guide/>
12. Firebase Pricing - Google, truy cập vào tháng 5 10, 2026, <https://firebase.google.com/pricing>
13. Supabase Storage or S3 over Firebase Storage? (Bandwidth cost breakdown) - Reddit, truy cập vào tháng 5 10, 2026, [https://www.reddit.com/r/iOSProgramming/comments/18fp2xa/supabase\\_storage\\_or\\_s3\\_over\\_firebase\\_storage/](https://www.reddit.com/r/iOSProgramming/comments/18fp2xa/supabase_storage_or_s3_over_firebase_storage/)
14. Is Google Drive really cheaper than S3 storage? : r/aws - Reddit, truy cập vào tháng 5 10, 2026, [https://www.reddit.com/r/aws/comments/1r10ppv/is\\_google\\_drive\\_really\\_cheaper\\_than\\_s3\\_storage/](https://www.reddit.com/r/aws/comments/1r10ppv/is_google_drive_really_cheaper_than_s3_storage/)
15. romajs/cloudinary-mock: Cloudinary mock implementation ... - GitHub, truy cập vào tháng 5 10, 2026, <https://github.com/romajs/cloudinary-mock>
16. Serve static files with a custom mock endpoint - Mockoon, truy cập vào tháng 5 10, 2026, <https://mockoon.com/tutorials/create-endpoint-serving-static-file/>
17. Node.js File Upload to a Local Server Or to the Cloud - Cloudinary, truy cập vào tháng 5 10, 2026, [https://cloudinary.com/blog/node\\_js\\_file\\_upload\\_to\\_a\\_local\\_server\\_or\\_to\\_the\\_cloud](https://cloudinary.com/blog/node_js_file_upload_to_a_local_server_or_to_the_cloud)
18. GPT-5.5 Model | OpenAI API, truy cập vào tháng 5 10, 2026, <https://developers.openai.com/api/docs/models/gpt-5.5>
19. Improving Structured Outputs in the Gemini API - Google Blog, truy cập vào tháng 5 10, 2026, <https://blog.google/innovation-and-ai/technology/developers-tools/gemini-api-structured-outputs/>
20. Gemini 3.1 Flash-Lite | Gemini API - Google AI for Developers, truy cập vào tháng 5 10, 2026, <https://ai.google.dev/gemini-api/docs/models/gemini-3.1-flash-lite>
21. Gemini 3.1 Flash-Lite | Generative AI on Vertex AI - Google Cloud Documentation, truy cập vào tháng 5 10, 2026,



<https://docs.cloud.google.com/vertex-ai/generative-ai/docs/models/gemini/3-1-flan-lite>

22. Structured outputs - generateContent API - Google AI for Developers, truy cập vào tháng 5 10, 2026, <https://ai.google.dev/gemini-api/docs/structured-output>
23. List of universities in Vietnam - Wikipedia, truy cập vào tháng 5 10, 2026, [https://en.wikipedia.org/wiki/List\\_of\\_universities\\_in\\_Vietnam](https://en.wikipedia.org/wiki/List_of_universities_in_Vietnam)
24. Ho Chi Minh City University of Technology - Wikipedia, truy cập vào tháng 5 10, 2026, [https://en.wikipedia.org/wiki/Ho\\_Chi\\_Minh\\_City\\_University\\_of\\_Technology](https://en.wikipedia.org/wiki/Ho_Chi_Minh_City_University_of_Technology)
25. Top Vietnam's Universities In Computer Science And IT-related Majors 2024: Centers Of Excellence For Future Software Engineering Leaders - Inspius | Hire & Manage Remote Tech Talent In Vietnam For Singapore Firms, truy cập vào tháng 5 10, 2026, <https://inspius.com/insights/best-universities-in-computer-science-vietnam-ranked/>
26. truy cập vào tháng 1 1, 1970, <https://topdev.vn/blog/viet-cv-cho-fresher-it/>
27. Using transformer-based models for Vietnamese language detection - PMC, truy cập vào tháng 5 10, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC12904414/>
28. An Investigation into the Common Pronunciation Mistakes of Vietnamese Students and Possible Solutions: A Case Study at a University in Hanoi - ResearchGate, truy cập vào tháng 5 10, 2026, [https://www.researchgate.net/publication/394717152\\_An\\_Investigation\\_into\\_the\\_Common\\_Pronunciation\\_Mistakes\\_of\\_Vietnamese\\_Students\\_and\\_Possible\\_Solutions\\_A\\_Case\\_Study\\_at\\_a\\_University\\_in\\_Hanoi](https://www.researchgate.net/publication/394717152_An_Investigation_into_the_Common_Pronunciation_Mistakes_of_Vietnamese_Students_and_Possible_Solutions_A_Case_Study_at_a_University_in_Hanoi)
29. An Investigation into the Common Pronunciation Mistakes of Vietnamese Students and Possible Solutions, truy cập vào tháng 5 10, 2026, <https://ijsshr.in/v8i7/Doc/26.pdf>
30. GitHub - BerriAI/litellm: Python SDK, Proxy Server (AI Gateway) to call 100+ LLM APIs in OpenAI (or native) format, with cost tracking, guardrails, loadbalancing and logging. [Bedrock, Azure, OpenAI, VertexAI, Cohere, Anthropic, Sagemaker, HuggingFace, VLLM, NVIDIA NIM], truy cập vào tháng 5 10, 2026, <https://github.com/BerriAI/litellm>
31. Evidently 0.7.17: open-source LLM tracing and dataset management, truy cập vào tháng 5 10, 2026, <https://www.evidentlyai.com/blog/open-source-llm-tracing>
32. Top Open-Source LLM Observability Tools in 2025 | by The Practical Developer | Medium, truy cập vào tháng 5 10, 2026, <https://medium.com/@thepracticaldeveloper/top-open-source-llm-observability-tools-in-2025-d2d5cbf4b932>
33. Synthetic Testing: What It Is & How It Works | Datadog, truy cập vào tháng 5 10, 2026, <https://www.datadoghq.com/knowledge-center/synthetic-testing/>
34. Synthetic Monitoring | Grafana Cloud documentation, truy cập vào tháng 5 10, 2026, <https://grafana.com/docs/grafana-cloud/testing/synthetic-monitoring/>